

DESARROLLO WEB INTEGRADO

Test-drive development (TDD)

Sesión 3

Ciclo VII: Agosto 2026

Mg. Ing. Richard Doann ASENCIOS VILCHEZ



Universidad
Tecnológica
del Perú

Unidad 1 API REST



inicio



Universidad
Tecnológica
del Perú

**“Primero define cómo
debe comportarse el
software; después
escribe el código que
lo hará posible”**

INICIO

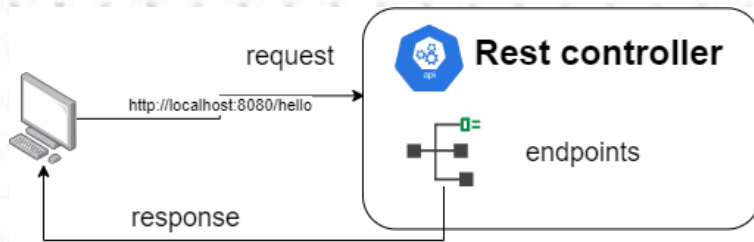
Dudas sobre la sesión anterior



INICIO

Dudas sobre la sesión anterior

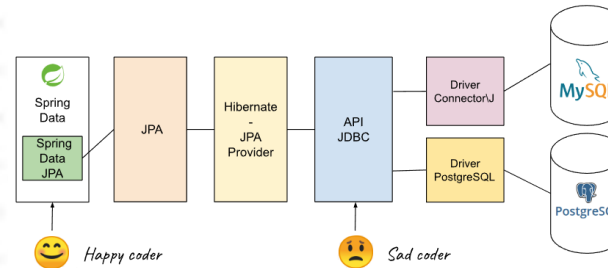
ENDPOINT



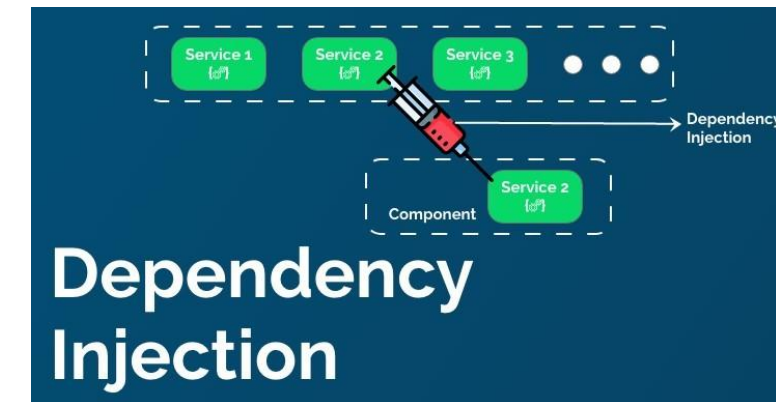
Explicación

- **@RestController**: Indica que esta clase manejará peticiones **HTTP** y devolverá respuestas en formato JSON o texto.
- **@RequestMapping("/api/usuarios")**: Define la ruta base de los endpoints de este controlador.
- **@GetMapping("/{id}")**: Define un endpoint para obtener un usuario por su **ID**.
- **@PathVariable**: Extrae el ID de la URL.
- **@PostMapping**: Define un endpoint para **crear un usuario**.
- **@RequestBody**: Recibe el cuerpo de la solicitud HTTP (usado en POST o PUT).

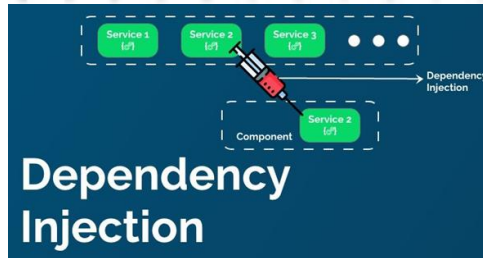
Configuración de endpoints y controladores



DEPENDENCY INJECTION

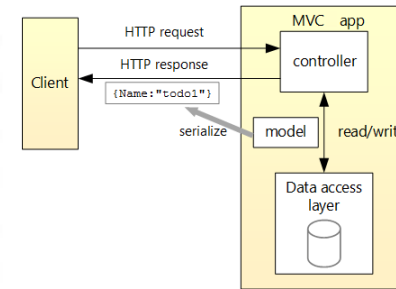


Conocimientos Previos



1.- Es una URL específica a la que los clientes pueden enviar solicitudes HTTP para interactuar con una aplicación web.

- a. RestController
- b. BackEnd
- c. GetMapping
- d. Controladores
- e. Endpoints

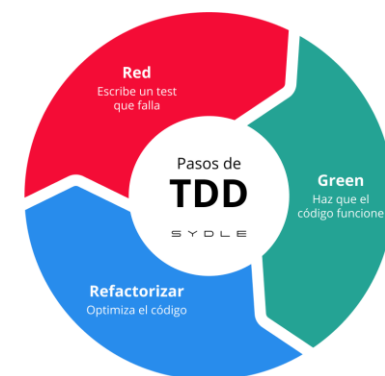


2.- Es un patrón de diseño que permite desacoplar componentes, mejorando la flexibilidad y el mantenimiento del código. En lugar de que los objetos creen sus propias dependencias, estas son proporcionadas por un contenedor de Inversión de Control (IoC) de Spring Boot.

- a. Inyección de dependencias
- b. BackEnd
- c. GetMapping
- d. Controladores
- e. Endpoints



Este tema servirá para integrar Test-Driven Development (TDD) en un entorno como Spring Boot, ya que este framework promueve el desarrollo basado en **buenas prácticas** de arquitectura y modularidad. Así mismo, TDD potencia esas virtudes y ayuda a construir aplicaciones robustas, mantenibles y bien diseñadas desde el inicio.



¿De que otra forma podemos utilizar lo aprendido?

¿Para que será útil lo que estamos aprendiendo en esta sección?

TEST-DRIVEN DEVELOPMENT



Test Driven Development (TDD)

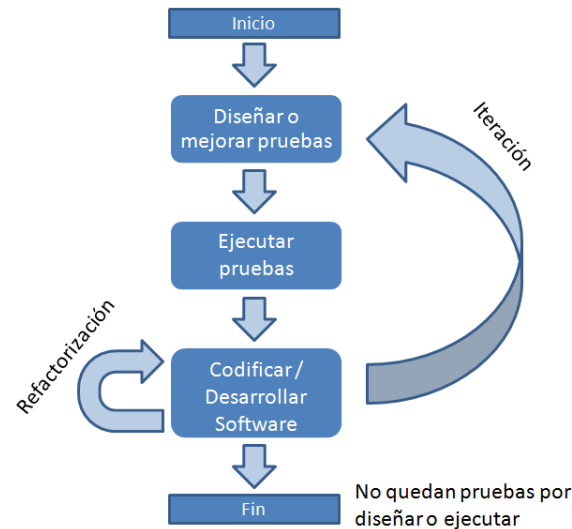
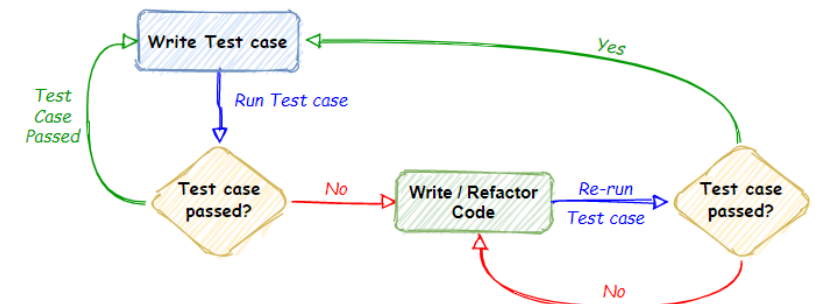


Imagen elaborada por oficinaproyectossoftware.blogspot.com

Test Drive Development



UTILIDAD

Logro de la sesión:



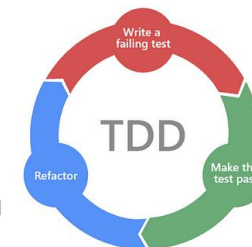
Al finalizar la sesión el estudiante utilizará Test-drive development (TDD) integrado a Spring Boot para promover las buenas practicas , a través de casos prácticos.



TEMA:

Test-drive development (TDD)

**TEST-DRIVEN
DEVELOPMENT**



Test-Driven Development (TDD)

Test-Driven Development (TDD), o Desarrollo Guiado por Pruebas, es una metodología de desarrollo de software en la que se escriben primero las pruebas (tests) antes de implementar el código funcional. El objetivo principal de TDD es garantizar que el **software funcione correctamente desde el principio** y que sea más fácil de mantener y refactorizar a lo largo del tiempo.



¿Cómo funciona TDD?

Se basa en un ciclo repetitivo de tres pasos, conocido como el ciclo **Red-Green-Refactor**:

1. **Red (Rojo)**: Escribes una **prueba unitaria** que falla porque la funcionalidad aún no está implementada.
2. **Green (Verde)**: Escribes el **código mínimo necesario** para que la prueba pase (sin preocuparte aún por la eficiencia o estilo).
3. **Refactor (Refactorizar)**: Limpias y mejoras el **código** manteniendo el comportamiento correcto (todas las pruebas deben seguir pasando).

Test-Driven Development (TDD)



Ventajas de TDD

- Mejora la calidad del software.
- Permite detectar errores antes.
- Facilita el cambio y mantenimiento del código.
- Fomenta un diseño más limpio y modular.
- Provee documentación viviente a través de los tests.

Desventajas (o desafíos)

- Requiere disciplina y práctica.
- Al principio, puede parecer más lento.
- No reemplaza otras formas de prueba (como pruebas de integración o funcionales).

Test-Driven Development (TDD)

Utilizar Test-Driven Development (TDD) en un entorno como Spring (especialmente con Spring Boot) es altamente beneficioso, ya que este framework promueve el **desarrollo basado en buenas prácticas de arquitectura y modularidad**. Integrar TDD potencia esas virtudes y ayuda a construir aplicaciones robustas, mantenibles y bien diseñadas desde el inicio.



Test-Driven Development (TDD)

Nos enfocamos particularmente en cómo las herramientas modernas como **JUnit 5 y Mockito** complementan prácticas como el Test-Driven Development (TDD) para mejorar la **calidad y la eficiencia del desarrollo**.

Propósito de las Pruebas de Software

El propósito fundamental de las pruebas de software es garantizar que las aplicaciones cumplan con los **requisitos funcionales y no funcionales**, y operen de manera eficiente en escenarios de uso real. Estas pruebas son cruciales para identificar bugs, mejorar la calidad del software y asegurar la satisfacción del usuario, sirviendo como una red de seguridad que permite a los desarrolladores realizar cambios y mejoras con confianza.



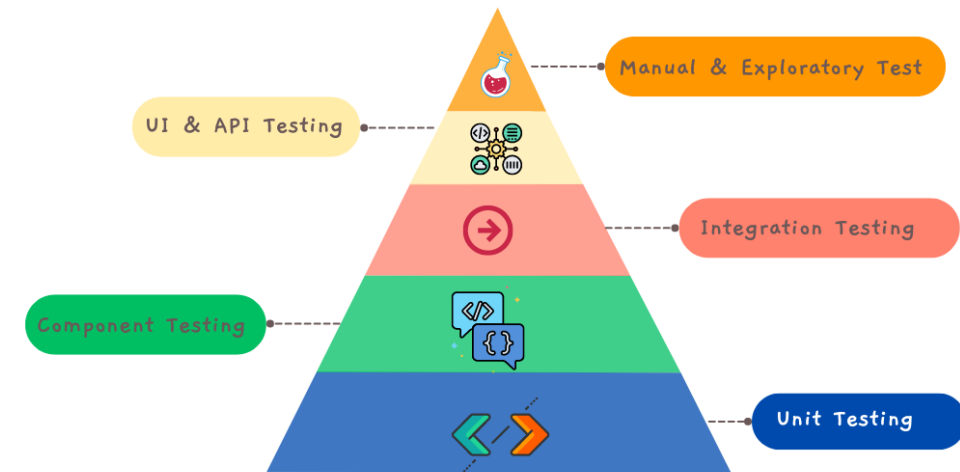
Test-Driven Development (TDD)

Jerarquía de Alcance de las Pruebas

Las pruebas de software se estructuran en distintos niveles para abordar diferentes aspectos del sistema:

- **Pruebas unitarias:** El nivel **base**, donde se verifica cada **componente o unidad** de manera aislada para asegurar que funcione correctamente según lo especificado.
- **Pruebas de integración:** Examinan la **interacción entre componentes y módulos** para asegurar que las unidades combinadas funcionen juntas según lo diseñado.
- **Pruebas de sistema:** Evalúan el comportamiento del sistema completo en un entorno que simula la producción.
- **Pruebas de UI y API:** Se centran en la funcionalidad expuesta a través de las interfaces de usuario y APIs.
- **Pruebas manuales y exploratorias:** Implican interacción humana para encontrar problemas que las pruebas automatizadas podrían no detectar.

JERARQUIA DE ALCANCE DE LAS PRUEBAS



Como se puede observar en la Figura, esta estructura piramidal asegura una cobertura completa, desde la validación de la lógica interna del código hasta la experiencia del usuario final.

TRANSFORMACIÓN

Integración de JUnit 5, Mockito y TDD

El corazón de una estrategia de prueba efectiva en Spring Boot reside en la integración de herramientas como **JUnit 5 y Mockito**, potenciadas por la metodología de Test-Driven Development (TDD). Estos componentes trabajan en conjunto para proporcionar un enfoque robusto y eficiente para asegurar la calidad del software.

JUnit 5

La última generación del framework de **pruebas unitarias para Java**, JUnit 5, introduce mejoras significativas que facilitan la escritura y ejecución de pruebas. Sus características permiten a los desarrolladores realizar pruebas más complejas y eficientes: Algunas de las **anotaciones y funcionalidades** más destacadas incluyen:

- **@Test**: Declara que un método es una prueba.
- **@ParameterizedTest**: Permite ejecutar una prueba con diferentes argumentos. Es necesario definir una fuente que proporcione los datos para cada invocación del método de prueba.

- **@RepeatedTest**: Facilita la ejecución repetida de una prueba un número determinado de veces.
- **@BeforeEach** / **@AfterEach**: Métodos que se ejecutan antes y después de cada prueba, respectivamente, útiles para preparar y limpiar condiciones.
- **@BeforeAll** / **@AfterAll**: Métodos que se ejecutan antes y después de todas las pruebas en una clase, ideal para configurar o desmontar recursos compartidos.

Mockito

Mockito, un framework de simulación para Java, es ampliamente utilizado para **testear clases con dependencias externas**. Permite a los desarrolladores simular esas dependencias eficazmente:

- **@Mock**: Crea y inyecta instancias simuladas (mocks). Se utiliza en lugar de Mockito.mock(), proporcionando una forma más clara y concisa de crear mocks.
- **@Spy**: Permite crear un 'spy' sobre un objeto real, lo que significa que se puede supervisar cómo se utilizan los métodos reales del objeto mientras se conserva su comportamiento original.
- **@InjectMocks**: Automatiza la inyección de mocks en el objeto bajo prueba, intentando la inyección a través de constructor, métodos setter o directamente en las propiedades.

Configuración del entorno de testing en Spring Boot

Dependencia en **pom.xml** (Maven)

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-test</artifactId>  
  <scope>test</scope>  
</dependency>
```

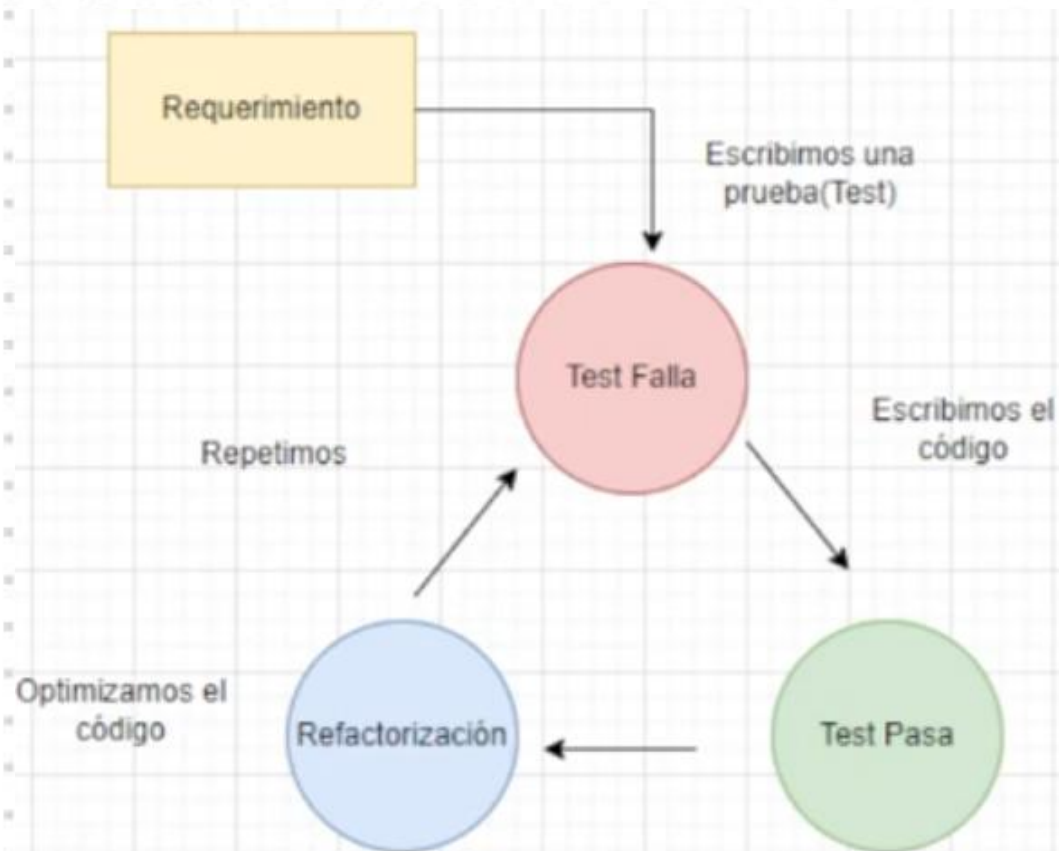
Herramientas incluidas

- **JUnit 5** – Framework de pruebas
- **Mockito** – Simulación de dependencias
- **Spring Test** – Soporte para pruebas con contexto Spring

Anotaciones clave

- @Test** – Marca métodos de prueba
- @SpringBootTest** – Carga el contexto completo
- @WebMvcTest** – Pruebas solo para controladores
- @DataJpaTest** – Pruebas para la capa de persistencia
- @MockBean, @Mock, @InjectMocks** – Para mocks

Test de capa web (controladores), sin levantar la base de datos ni todo el contexto de Spring Boot → rápido y eficaz para validar APIs REST.



Requerimientos para API en Spring Boot con Test-Driven Development (TDD)

Objetivo

Desarrollar una API RESTful utilizando Spring Boot siguiendo la metodología de Test-Driven Development (TDD). El objetivo principal es garantizar la calidad del código y la funcionalidad de la API mediante pruebas automatizadas desde el principio del desarrollo.

Funcionalidades Principales

1. Endpoint para Crear un Nuevo Recurso

Método: POST

Ruta: /api/recurso/save

Parámetros de Entrada:

Cuerpo de la solicitud con los datos del nuevo recurso en formato JSON.

Respuestas:

- 201 Created: Recurso creado exitosamente.

2. Endpoint para Obtener la lista de recursos

Método: GET

Ruta: /api/recurso

Respuestas:

- 200 OK: Recurso encontrado y devuelto en formato JSON con 5 elementos.

Paso 1: Crear proyecto

Para comenzar, crea un nuevo proyecto Spring Boot. Esto se puede hacer fácilmente utilizando Spring Initializr, una herramienta en línea que permite configurar proyectos Spring básicos de manera personalizada.

1. Accede a **Spring Initializr**: Ve a start.spring.io.
2. Selecciona las Dependencias del Proyecto: Añade las siguientes dependencias para asegurarte de que tu proyecto tenga todo lo necesario para el desarrollo y pruebas:
 - **Spring Web**: Para construir servicios web, incluyendo aplicaciones RESTful utilizando Spring MVC.
 - **Lombok**: Para reducir el código boilerplate en Java mediante anotaciones.

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

Paso 2: Crear una Entidad (clase Local)

```
import lombok.AllArgsConstructor;
import lombok.Builder;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@NoArgsConstructor
@AllArgsConstructor
@Builder
public class Local {
    private Long id;
    private String name;
    private String floor;
}
```

Esta clase representa el modelo de datos de un **local**.

Por ejemplo, un objeto podría representar:

- id = 1
- name = "PetShop"
- floor = "Third Floor"

Paso 3: Crear clase para test del controlador (Clase LocalControllerTest)

Clase que realiza pruebas sobre un controlador Spring MVC.
Spring inyecta automáticamente un objeto **MockMvc**. Este objeto permite simular solicitudes (get, post, put...).

Primera prueba: findAllLocals()

- Primera validación: HTTP 200
- Segunda validación: tamaño del JSON

Segunda prueba: registerLocal()

- Primera validación del POST (isCreated)
- Segunda validación del POST

```
@WebMvcTest
@AutoConfigureMockMvc
public class LocalControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void findAllLocals() throws Exception {
        mockMvc.perform(MockMvcRequestBuilders.get("/api/recurso"))
            .andExpect(MockMvcResultMatchers.status().isOk())
            .andExpect(MockMvcResultMatchers.jsonPath("$", Matchers.hasSize(5)));
    }

    @Test
    void registerLocal() throws Exception {
        Local local = Local.builder()
            .id(1L)
            .name("PetShop")
            .floor("Third Floor")
            .build();
        mockMvc.perform(MockMvcRequestBuilders.post("/api/recurso/save")
            .contentType(MediaType.APPLICATION_JSON)
            .content(asJsonString(local)))
            .andExpect(MockMvcResultMatchers.status().isCreated())
            .andExpect(MockMvcResultMatchers.content().string("Recurso creado exitosamente.));

    }

    public static String asJsonString(final Object obj){
        try{
            return new ObjectMapper().writeValueAsString(obj);
        }catch (Exception e){
            throw new RuntimeException();
        }
    }
}
```

Output - Run () Test Results ×

Semana3:test3:jar:0.0.1-SNAPSHOT (Unit) × semana3:test1:jar:0.0.1-SNAPSHOT (Unit) ×

Tests passed: 0.00 %

No test passed, 1 test failed. (2.549 s)

- semana3.test.controller.LocalControllerTest Failed
 - findAllLocals Failed: Status expected:<200> but was:<404>

Error message = No static resource api/recurso.

Headers = [Vary:"Origin", "Access-Control-Request-Method", "

Content type = null

Body =

Forwarded URL = null

Redirected URL = null

Cookies = []

semana3:test1:jar:0.0.1-SNAPSHOT (Unit) ×

Tests passed: 0.00 %

No test passed, 1 test failed. (5.292 s)

- semana3.test.controller.LocalControllerTest Failed
 - registerLocal Failed: Status expected:<201> but was:<404>

MockHttpServletRequest:

HTTP Method = POST

Request URI = /api/recurso/save

Parameters = {}

Headers = [Content-Type:"applicat

Body = {"id":1,"name":"PetShop

Session Attrs = {}



Implementación Práctica

Paso 4: Crear clase para el controlador (Clase LocalController)

```
@RestController
@RequestMapping("api/recurso")
public class LocalController {

    @PostMapping("/save")
    ResponseEntity<String> registerLocal(@RequestBody Local local) {
        // TODO: Lógica para guardar el objeto Local en tu servicio
        String message = "Recurso creado exitosamente.";
        return ResponseEntity.status(HttpStatus.CREATED).body(message);
    }

    @GetMapping
    ResponseEntity<List<Local>> getAllLocals() {
        List<Local> localList = new ArrayList<>();

        Local local1 = new Local();
        local1.setId(1L);
        local1.setName("Local UNO");
        local1.setFloor("Primer Piso");
        localList.add(local1);

        Local local2 = new Local();
        local2.setId(2L);
        local2.setName("Local DOS");
        local2.setFloor("Segundo Piso");
        localList.add(local2);

        Local local3 = new Local();
        local3.setId(3L);
        local3.setName("Local TRES");
        local3.setFloor("Tercer Piso");
        localList.add(local3);

        Local local4 = new Local();
        local4.setId(4L);
        local4.setName("Local CUATRO");
        local4.setFloor("Cuarto Piso");
        localList.add(local4);

        Local local5 = new Local();
        local5.setId(5L);
        local5.setName("Local CINCO");
        local5.setFloor("Quinto Piso");
        localList.add(local5);

        return ResponseEntity.ok(localList);
    }
}
```



Paso : Optimización del Código (Clase LocalController)

```
@RestController
@RequestMapping("api/recurso")
public class LocalController {

    @PostMapping("/save")
    ResponseEntity<String> registerLocal(@RequestBody Local local) {
        // TODO: Lógica para guardar el objeto Local en tu servicio
        String message = "Recurso creado exitosamente.";
        return ResponseEntity.status(HttpStatus.CREATED).body(message);
    }

    @GetMapping
    ResponseEntity<List<Local>> getAllLocals() {
        List<Local> localList = new ArrayList<>();

        for(long i = 1; i<=3; i++){
            Local local = new Local();
            local.setId(i);
            local.setName("Local " + i);
            local.setFloor("Floor " + i);
            localList.add(local);
        }

        return ResponseEntity.ok(localList);
    }
}
```

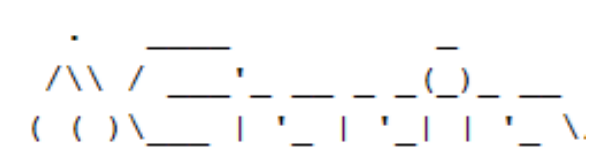
Semana3:test3:jar:0.0.1-SNAPSHOT (Unit) ×

Tests passed: 100.00 %

The test passed. (2.941 s)

- ✓ Semana3.test3.controller.LocalControllerTest passed
 - ✓ findAllLocals passed (0.814 s)

20:14:50.252 [main] INFO or
20:14:50.463 [main] INFO or



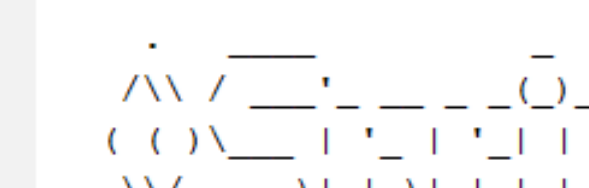
Semana3:test3:jar:0.0.1-SNAPSHOT (Unit) ×

Tests passed: 100.00 %

The test passed. (2.925 s)

- ✓ Semana3.test3.controller.LocalControllerTest passed
 - ✓ registerLocal passed (0.936 s)

20:17:00.348 [main] INF
20:17:00.516 [main] INF





¿Que hemos aprendido hoy?

1. Test-Driven Development es una metodología de desarrollo de software que consiste, básicamente, en escribir antes los tests que la funcionalidad.
2. Estas pruebas unitarias ilustran un enfoque efectivo para validar la funcionalidad de servicios REST en Spring, utilizando JUnit y Mockito para simular dependencias y verificar comportamientos sin necesidad de cargar el contexto completo de Spring. Este método no solo mejora la velocidad de las pruebas, sino que también facilita un proceso de desarrollo más ágil y enfocado.



**Universidad
Tecnológica
del Perú**